

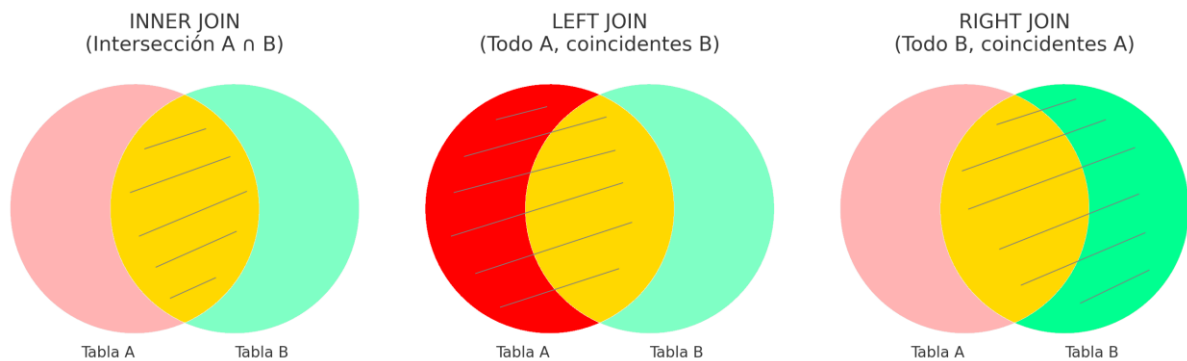
# Módulo 5: JOINS - Uniendo Tablas

## JOINS: Uniendo Tablas

Esta práctica está dividida en dos partes:

- Tipos de JOINS (**INNER**, **LEFT**, **RIGHT**)
- Consultas compleja con múltiples **JOIN**

## Tipos de JOIN: INNER, LEFT y RIGHT



En una base de datos relacional, la información no se guarda toda junta en una sola tabla, sino dividida en varias tablas relacionadas entre sí. Esto se hace para evitar la duplicación de datos, mejorar la organización y facilitar el mantenimiento de la base.

Por ejemplo:

- Los alumnos están en una tabla.
- Las carreras en otra.
- Las asignaturas en otra.

Cada tabla guarda un tipo específico de información. Pero cuando queremos hacer una consulta que combine datos de varias tablas, necesitamos reconstruir esas relaciones. Para eso usamos los comandos JOIN, que permiten:

- Conectar una tabla con otra a través de un campo en común (por ejemplo, el `id_carrera` en alumnos coincide con el `id` en carreras, se vincula por lo general una ).
- Obtener información combinada que tenga sentido para el análisis o la presentación de resultados (por ejemplo: “mostrar el nombre del alumno y el nombre de su carrera”).

El uso de JOIN es lo que hace útil y potente a una base relacional, porque permite consultar datos dispersos de forma coherente y estructurada.

Esta práctica está centrada en los tres tipos más importantes:

INNER JOIN  
LEFT JOIN  
RIGHT JOIN

Trabajaremos con las tablas existentes en la base *gestionacademica*:

*alumnos*  
*carreras*  
*asignaturas*

**Datos de ejemplo sugeridos** (insertar antes de comenzar, verificar que no haya duplicación de claves primarias)

**Tabla *alumnos***

| id | nombre | apellido | edad | id_carrera |
|----|--------|----------|------|------------|
| 1  | Ana    | Gómez    | 20   | 1          |
| 2  | Juan   | Pérez    | 21   | 2          |
| 3  | Sol    | Martínez | 22   | NULL       |

**Tabla *carreras***

| id | nombre_carrera |
|----|----------------|
| 1  | Informática    |
| 2  | Electrónica    |
| 3  | Química        |

## INNER JOIN – Solo coincidencias

**SELECT** alumnos.nombre, carreras.nombre\_carrera

**FROM** alumnos

**INNER JOIN** carreras **ON** alumnos.id\_carrera = carreras.id;

Esta consulta muestra el nombre de cada alumno junto con el nombre de su carrera, **pero solo si el alumno tiene una carrera asignada y esa carrera existe.**

**SELECT** alumnos.nombre, carreras.nombre\_carrera → Muestra el nombre del alumno y el nombre de su carrera.

**FROM** alumnos → Se parte de la tabla *alumnos*.

**INNER JOIN** carreras → Se une con la tabla *carreras*.

**ON** alumnos.id\_carrera = carreras.id → Se especifica que deben coincidir el *id\_carrera* del alumno con el *id* de la carrera.

**Resultado:**

| nombre | nombre_carrera |
|--------|----------------|
| Ana    | Informática    |
| Juan   | Electrónica    |

**EJERCICIOS A REALIZAR**

**Generar y ejecutar las consultas solicitadas, mostrando captura de pantalla de la consulta y de su resultado.**

1. Mostrar el nombre completo del alumno y la duración de su carrera.
2. Listar alumnos que pertenecen a carreras del departamento 'Exactas'.

**LEFT JOIN – Todo desde la izquierda, coincidencias si existen**

```
SELECT alumnos.nombre, carreras.nombre_carrera  
FROM alumnos  
LEFT JOIN carreras ON alumnos.id_carrera = carreras.id;
```

Esta consulta muestra **todos los alumnos**, independientemente de si tienen o no una carrera asignada. Si un alumno **no tiene carrera**, en la columna **nombre\_carrera** aparecerá **NULL**.

**SELECT alumnos.nombre, carreras.nombre\_carrera** → Muestra el nombre de cada alumno y el nombre de su carrera (si tiene).

**FROM alumnos** → Se parte de la tabla **alumnos**.

**LEFT JOIN carreras** → Se une con la tabla **carreras**, pero **conservando todos los registros de la tabla izquierda (alumnos)**.

**ON alumnos.id\_carrera = carreras.id** → Se comparan los campos **id\_carrera** (de **alumnos**) con **id** (de **carreras**).

**Resultado:**

| nombre | nombre_carrera |
|--------|----------------|
| Ana    | Informática    |
| Juan   | Electrónica    |
| Sol    | NULL           |

"Sol" aparece en el resultado aunque no tenga carrera asignada (`id_carrera = NULL`), gracias al uso de `LEFT JOIN`.

## EJERCICIOS A REALIZAR

Generar y ejecutar las consultas solicitadas, mostrando captura de pantalla de la consulta y de su resultado.

3. Mostrar todos los alumnos junto con el nombre y duración de su carrera (si la tienen).
4. Mostrar los nombres de alumnos y el departamento de su carrera, incluyendo los que no tienen carrera asignada.

## RIGHT JOIN – Todo desde la derecha, coincidencias si existen

```
SELECT alumnos.nombre, carreras.nombre_carrera  
FROM alumnos  
RIGHT JOIN carreras ON alumnos.id_carrera = carreras.id;
```

### Explicación:

Esta consulta muestra **todas las carreras**, incluso si no tienen alumnos asociados. Si una carrera **no tiene ningún alumno**, en la columna `nombre` aparecerá `NULL`.

`SELECT alumnos.nombre, carreras.nombre_carrera` → Muestra el nombre del alumno (si existe) y el nombre de la carrera.

`FROM alumnos` → Se parte de la tabla `alumnos`, aunque el protagonismo lo tiene `carreras` por el `RIGHT JOIN`.

`RIGHT JOIN carreras` → Se conserva **toda la información de la tabla derecha** (`carreras`), uniendo con la tabla `alumnos`.

`ON alumnos.id_carrera = carreras.id` → Se unen los datos donde el `id_carrera` del alumno coincide con el `id` de la carrera.

### Resultado:

| nombre | nombre_carrera |
|--------|----------------|
| Ana    | Informática    |
| Juan   | Electrónica    |
| NULL   | Química        |

## EJERCICIOS A REALIZAR

Generar y ejecutar las consultas solicitadas, mostrando captura de pantalla de la consulta y de su resultado.

5. Mostrar todas las carreras y el nombre de sus alumnos si los tienen.
6. Mostrar todas las carreras junto con el departamento y los apellidos de los alumnos (si hay).

## Consultas con múltiples JOINS

### Introducción

En los ejercicios anteriores ya trabajamos con datos de ejemplo sobre las tablas **alumnos**, **carreras** y **asignaturas**.

Ahora vamos a **agregar una nueva tabla** llamada **matriculas** que nos permitirá vincular alumnos con asignaturas y registrar sus calificaciones. Esta tabla representa el acto de inscripción a una materia y es común en cualquier sistema académico real.

Esta tabla no estaba incluida en la base original **gestionacademica**, pero es una extensión lógica y necesaria para esta práctica.

### Creamos la tabla **matriculas**

```
CREATE TABLE IF NOT EXISTS matriculas (  
  id_alumno INT NOT NULL,  
  id_asignatura INT NOT NULL,  
  calificacion DECIMAL(3,1),  
  PRIMARY KEY (id_alumno, id_asignatura),  
  FOREIGN KEY (id_alumno) REFERENCES alumnos(id),  
  FOREIGN KEY (id_asignatura) REFERENCES asignaturas(id)  
);
```

### Explicación:

**FOREIGN KEY (id\_alumno) REFERENCES alumnos(id)** Esto indica que el campo **id\_alumno** de la tabla **matriculas** está **relacionado** con el campo **id** de la tabla **alumnos**. Es decir, **solo se puede insertar un id\_alumno si ese valor ya existe en la tabla alumnos**.

**FOREIGN KEY (id\_asignatura) REFERENCES asignaturas(id)** Lo mismo ocurre con **id\_asignatura**: debe coincidir con un valor ya existente en el campo **id** de la tabla **asignaturas**.

### ¿Para qué sirve **REFERENCES**?

**Establece relaciones entre tablas** Define que ciertos datos en una tabla deben coincidir con datos de otra tabla.

**Garantiza integridad referencial** Evita que se puedan insertar datos "huérfanos". Por ejemplo, no podés matricular a un alumno que no existe.

**Facilita consultas con JOINS** Al tener estas relaciones claras, es más fácil hacer consultas entre varias tablas sabiendo cómo están conectadas.

### ¿Qué pasa si no existe el valor referenciado?

Por ejemplo, si intentás hacer esto:

```
INSERT INTO matriculas (id_alumno, id_asignatura, calificacion)
VALUES (999, 1, 8.0);
```

Y no existe un alumno con **id = 999**, la base de datos **rechazará la operación**. Esto protege la coherencia de los datos.

**Insertamos datos de ejemplo en la tabla **matriculas**** (basados en los datos de ejemplos que habíamos insertado en los puntos anteriores)

```
INSERT INTO matriculas (id_alumno, id_asignatura, calificacion) VALUES
(1, 1, 9.0), -- Ana en Programación I
(1, 2, 8.0), -- Ana en Bases de Datos
(2, 3, 7.5), -- Juan en Electrónica I
(3, 1, 6.5); -- Sol en Programación I
```

### **Ejecutamos la Consulta de ejemplo con múltiples JOIN**

```
SELECT
a.nombre,
a.apellido,
c.nombre_carrera,
asig.nombre AS asignatura,
m.calificacion
FROM alumnos a
INNER JOIN carreras c ON a.id_carrera = c.id
INNER JOIN matriculas m ON a.id = m.id_alumno
INNER JOIN asignaturas asig ON m.id_asignatura = asig.id
WHERE c.nombre_carrera = 'Informática'
ORDER BY a.apellido, a.nombre, asig.nombre;
```

## ¿Qué hace esta consulta?

Esta consulta busca mostrar información detallada de los **alumnos que cursan carreras de Informática**, incluyendo sus calificaciones en asignaturas. Para lograrlo, se combinan **cuatro tablas** usando **INNER JOIN**. Veamos cada parte:

**alumnos a**: Se da el alias **a** a la tabla **alumnos**.

**carreras c**: Alias **c** para la tabla **carreras**.

**matriculas m**: Alias **m** para la tabla que vincula alumnos y asignaturas.

**asignaturas asig**: Alias **asig** para la tabla que guarda información de cada asignatura.

Los alias son abreviaturas de los nombres de las tablas, se utilizan para evitar escribir muchas veces el nombre completo de la tabla, reduciendo la longitud de la consulta y mejorando la legibilidad.

## ¿Cómo se unen las tablas?

**a.id\_carrera = c.id**: vincula cada alumno con su carrera.

**a.id = m.id\_alumno**: relaciona al alumno con sus asignaturas inscritas.

**m.id\_asignatura = asig.id**: relaciona la matrícula con la asignatura concreta.

**WHERE c.nombre\_carrera = 'Informática'**: Esto **filtra** los resultados para mostrar **solo los alumnos que están en la carrera 'Informática'**. Verificar que se usen comillas simples ('), no tipográficas (").

## Resultado esperado:

| nombre | apellido | nombre_carrera | asignatura     | calificación |
|--------|----------|----------------|----------------|--------------|
| Ana    | Gómez    | Informática    | Bases de Datos | 8.0          |
| Ana    | Gómez    | Informática    | Programación I | 9.0          |



## Observaciones:

**Sol** no tiene carrera asignada, así que queda fuera por el **INNER JOIN** con **carreras**.

**Juan** cursa Electrónica, así que es descartado por la cláusula **WHERE**.

Solo **Ana** está en la carrera de Informática y tiene registros en **matriculas** con asignaturas correspondientes.

## EJERCICIOS A REALIZAR

**Generar y ejecutar las consultas solicitadas, mostrando captura de pantalla de la consulta y de su resultado.**

7. Ejecutar la consulta del ejemplo. Mostrar captura de la consulta y del resultado.
8. Realizar una consulta que muestre el nombre y apellido de los alumnos, el nombre de la carrera que cursan, el nombre de las asignaturas en las que están matriculados y la calificación obtenida, solo si la asignatura pertenece a la misma carrera que cursa el alumno.

Pista: Tanto la tabla alumnos como la tabla asignaturas tienen un campo id\_carrera. Solo deben mostrarse las asignaturas que pertenezcan a la misma carrera que cursa el alumno (coincidencia entre IDs a usar en WHERE).